

AWS & Gen AI Interview Cheat Sheet

| Interview Prep — Gen AI Role | Core AWS · Gen AI Services · Key Concepts · Interview Q&A

Table of Contents

- 1. [Core AWS Services](#)
- 2. [Compute & ML Infrastructure](#)
- 3. [Gen AI Specific AWS Services](#)
- 4. [Amazon Bedrock — Deep Dive](#)
- 5. [Key Gen AI Concepts](#)
- 6. [Common Interview Q&A](#)

1. Core AWS Services

| Must-know foundational services for any AWS role. Even Gen AI interviews test these.

Storage

Service	What It Is	Key Use in Gen AI
Amazon S3	Object storage for any amount of data	Data lakes, ML training datasets, model artifacts, logs
Amazon EBS	Block storage volumes attached to EC2	High-speed storage for GPU instances during training
Amazon EFS	Managed shared file system (NFS)	Shared storage across multiple training nodes
Amazon FSx for Lustre	High-performance parallel file system	Distributed LLM training across GPU clusters

Databases

Service	What It Is	Key Use in Gen AI
Amazon RDS / Aurora	Managed relational DB (MySQL, PostgreSQL)	Aurora PostgreSQL + pgvector for storing embeddings
Amazon DynamoDB	Serverless NoSQL key-value store	Conversation history, session state, feature stores

Service	What It Is	Key Use in Gen AI
Amazon OpenSearch	Managed Elasticsearch/OpenSearch	k-NN vector search for RAG pipelines
Amazon ElastiCache	Managed Redis/Memcached	Semantic caching of LLM responses

Networking & Integration

Service	What It Is	Key Use in Gen AI
Amazon VPC	Isolated virtual network	Secure Gen AI deployments, private Bedrock endpoints
API Gateway	Create and manage REST/WebSocket APIs	Expose LLM inference endpoints with auth & rate limiting
Amazon SQS	Message queuing service	Async Gen AI pipelines, batch inference job queuing
Amazon SNS	Pub/sub notification service	Fan-out events for multi-system Gen AI workflows
AWS Step Functions	Workflow orchestration	Orchestrate multi-step Gen AI pipelines

Security & Identity

Service	What It Is	Key Use in Gen AI
AWS IAM	Identity & Access Management	Least-privilege access to Bedrock models and S3 datasets
AWS KMS	Encryption key management	Encrypt fine-tuned models and vector stores at rest
AWS Secrets Manager	Secure secret storage	Store and rotate API keys for third-party LLM services
AWS CloudTrail	API call audit logs	Compliance auditing of all Bedrock/SageMaker API calls
AWS GuardDuty	ML-based threat detection	Detect anomalous access patterns to Gen AI resources

Developer & DevOps

Service	What It Is	Key Use in Gen AI
AWS Lambda	Serverless functions	Bedrock Agent action handlers, RAG pipeline triggers
Amazon CloudWatch	Monitoring, logging, alerting	Monitor token usage, model latency, error rates

Service	What It Is	Key Use in Gen AI
Amazon ECR	Container registry	Store Docker images for custom inference servers
Amazon ECS / EKS	Managed containers / Kubernetes	Deploy and auto-scale LLM inference microservices
AWS CDK / CloudFormation	Infrastructure as Code	Reproducible Gen AI infrastructure deployments

2. Compute & ML Infrastructure

EC2 GPU Instance Types

Instance Family	GPU	Best For
p3	NVIDIA V100	LLM training (older generation)
p4d / p4de	NVIDIA A100	Large-scale distributed LLM training
p5	NVIDIA H100	State-of-the-art LLM training
g4dn	NVIDIA T4	Cost-efficient inference
g5	NVIDIA A10G	Mid-tier inference & fine-tuning
g6	NVIDIA L40S	Latest-gen cost-efficient inference
trn1	AWS Trainium	Purpose-built training — ~50% cheaper than GPU
inf2	AWS Inferentia2	Purpose-built inference — ~40% cheaper per request

Tip: For interviews, know that **Trainium** = **training**, **Inferentia** = **inference**, and both are AWS's proprietary chips designed to reduce costs vs NVIDIA GPUs.

Amazon SageMaker — Component Breakdown

SageMaker is AWS's end-to-end ML platform. Know when to use it vs Bedrock.

Rule of thumb: Bedrock = use existing FMs via API. SageMaker = train/fine-tune/deploy custom models.

Component	What It Does
SageMaker Studio	Web-based ML IDE with notebooks, experiment tracking, MLflow integration
Training Jobs	Managed distributed training. Supports PyTorch, TF, Hugging Face, FSDP, DeepSpeed
SageMaker Pipelines	CI/CD for ML — define DAG pipelines: data → train → evaluate → deploy
SageMaker Endpoints	Deploy models as REST APIs (real-time, batch, async). LMI containers for multi-GPU LLMs
SageMaker JumpStart	One-click deploy of 700+ models: Llama 3, Mistral, Falcon, Stable Diffusion. Supports QLoRA fine-tuning
SageMaker Clarify	Bias detection and explainability for ML models
SageMaker Model Monitor	Detect data drift and model quality degradation in production
SageMaker Feature Store	Centralized store for ML features used across training and inference
SageMaker Ground Truth	Data labeling service with human annotators for RLHF workflows

3. Gen AI Specific AWS Services

Amazon Bedrock

AWS's flagship Gen AI service. Access Claude, Llama, Titan, Mistral, Stable Diffusion and more via a single API. No GPU management. Pay-per-token.

Key capabilities: Knowledge Bases (managed RAG), Agents (agentic AI), Guardrails (safety), Custom Models (fine-tuning), Model Evaluation.

See Section 4 for full deep dive.

Amazon Q Business

Fully managed enterprise AI assistant. Connects to 40+ data sources (S3, Confluence, SharePoint, Salesforce, ServiceNow). Provides answers grounded in company data with source citations. Admin controls for content filtering and topic blocking. Uses IAM Identity Center for per-user access control — users only see answers from documents they're allowed to access.

When to recommend: Enterprises that want an internal chatbot without building RAG from scratch.

Amazon Q Developer

AI coding assistant (GitHub Copilot competitor). Available as IDE plugin (VS Code, IntelliJ, JetBrains) and in the AWS Console. Key features:

- Code generation, explanation, and debugging
- `/transform` — automated legacy Java migration to modern versions
- Security vulnerability scanning
- Unit test generation
- CLI natural language to command translation

Amazon Lex

Managed chatbot service (powers Alexa). Build conversational interfaces with intents, slots, and fulfillment. Now integrates with Bedrock to power the NLU layer with foundation models. Used in contact center automation with Amazon Connect.

Amazon Transcribe / Polly / Translate

Service	Function	Gen AI Use Case
Transcribe	Speech-to-text	Voice input for Gen AI chatbots
Polly	Text-to-speech	Voice output for Gen AI assistants
Translate	Neural machine translation	Multilingual Gen AI applications

Amazon Rekognition / Textract

Service	Function	Gen AI Use Case
Rekognition	Image/video analysis (faces, objects, labels)	Vision input for multimodal RAG pipelines
Textract	Extract text and tables from documents/PDFs	Document ingestion for Knowledge Bases

Amazon Kendra

ML-powered enterprise search. Understands natural language queries. Supports 40+ connectors. Now serves as a retrieval backend for Bedrock Knowledge Bases, offering a higher-quality alternative to pure vector search for document-heavy use cases.

PartyRock

No-code/low-code Gen AI app builder powered by Bedrock. Drag-and-drop interface to build Gen AI apps. Great for proof-of-concepts, demos, and non-technical users experimenting with LLMs. Share apps via public URL.

4. Amazon Bedrock — Deep Dive

Available Foundation Models

Provider	Models	Best For	Modalities
Anthropic	Claude 3.5 Sonnet, Claude 3 Haiku, Claude 3 Opus	Complex reasoning, 200K context, coding	Text + Vision
Meta	Llama 3.1 (8B, 70B, 405B), Llama 3.2	Open-source, customization, on-premise portability	Text + Vision
Mistral AI	Mistral 7B, Mixtral 8x7B, Mistral Large	Fast inference, multilingual, low-cost	Text
Amazon Titan	Titan Text, Titan Embeddings v2, Titan Image	AWS-native, embeddings for RAG	Text + Image + Embeddings
Stability AI	SDXL 1.0, SD3	Image generation	Image
Cohere	Command R, Command R+, Embed v3	RAG-optimized, enterprise search, reranking	Text + Embeddings
AI21 Labs	Jamba, Jurassic-2	Long context, structured generation	Text

Bedrock Knowledge Bases (Managed RAG)

Fully managed RAG pipeline. Connect data sources → automatic chunking → embedding → vector storage → retrieval.

Supported data sources: S3, Web crawler, Confluence, SharePoint, Salesforce

Supported vector stores:

- Amazon OpenSearch Serverless
- Amazon Aurora PostgreSQL (pgvector)
- Redis Enterprise Cloud
- Pinecone
- MongoDB Atlas

Chunking strategies: Fixed-size, Semantic, Hierarchical

Default embedding model: Amazon Titan Embeddings v2

Bedrock Agents

Build autonomous AI agents that plan, use tools, and take multi-step actions.

How it works (ReAct loop):

1. User sends task
2. Agent **Reasons** — what needs to be done?
3. Agent **Acts** — calls a Lambda function (action group)
4. Agent **Observes** — reads the result
5. Repeat until task is complete
6. Return final answer

Key concepts:

- **Action Groups** — Lambda functions with OpenAPI schema describing available actions
- **Knowledge Base attachment** — agent can retrieve from RAG for factual grounding
- **Multi-agent** — supervisor agent delegates to specialized sub-agents
- **Session management** — maintains conversation context across turns

Bedrock Guardrails

Apply safety controls to any FM on Bedrock.

Guardrail Feature	What It Does
Content filtering	Block hate speech, violence, sexual content, misconduct
Topic denial	Block off-topic questions (e.g., "don't answer competitor questions")
PII redaction	Detect and mask names, SSN, credit cards, email addresses
Grounding check	Detect when responses aren't supported by retrieved context (hallucination)
Word filters	Block specific words or phrases
Sensitive information	Custom regex patterns for domain-specific PII

Bedrock Custom Models (Fine-Tuning)

Type	Description	Use Case
Fine-tuning	Supervised training on labeled JSONL examples. Adjusts model weights	Consistent format, domain tone, specific task behavior

Type	Description	Use Case
Continued pre-training	Train on large unlabeled domain corpus	Domain language adaptation (medical, legal, finance)
Model distillation	Use a large teacher model to train a smaller student model	Cost reduction while preserving quality

Bedrock Inference Modes

Mode	Description	Use Case	Cost
On-Demand	Pay per token, no commitment	Variable/unpredictable traffic	Highest per-token
Provisioned Throughput	Reserved capacity (1–6 month commitment)	Consistent high-volume production	Lower per-token
Batch Inference	Process large datasets asynchronously	Bulk processing, offline analytics	Lowest cost
Cross-Region Inference	Automatically route to available region	High availability, regional failover	Standard + data transfer

Bedrock Model Evaluation

Compare multiple FMs on your own test prompts and expected outputs.

- **Automatic metrics:** ROUGE, BERTScore, accuracy, robustness
- **Human evaluation:** Use internal teams or Amazon Mechanical Turk as the labeling workforce
- **Use it to:** Pick the right model for your use case before committing to production

5. Key Gen AI Concepts

RAG — Retrieval Augmented Generation

Architecture flow:

- User Query
- Embed query (Titan Embeddings / Cohere Embed)
 - Vector similarity search (OpenSearch / Aurora pgvector / Pinecone)
 - Retrieve top-K chunks (K = 3–5 typically)
 - Inject chunks into prompt as context
 - LLM generates grounded response
 - Return answer with source citations

Key RAG concepts:

Concept	Description
Chunking	Breaking documents into pieces. Fixed-size (512–1024 tokens), semantic, or hierarchical. Chunk size affects retrieval quality.
Embeddings	Dense vector representations. Similar meaning = similar vectors. Common: Titan Embeddings (1536D), Cohere Embed v3
Vector Search	k-NN algorithms (HNSW, IVF) to find nearest vectors. Approximate NN for scale.
Hybrid Search	Combine vector search with BM25 keyword search. Often better than pure vector in production.
Reranking	Second-pass model (Cohere Rerank, cross-encoders) to reorder initial results. Improves precision significantly.
Context window	Retrieved chunks + system prompt + user query must fit within model's token limit

Prompt Engineering Techniques

Technique	Description	When to Use
Zero-Shot	Ask directly without examples	Simple tasks, capable models (Claude 3.5)
Few-Shot	Provide 2–5 examples in prompt	Specific output formats, classification
Chain-of-Thought (CoT)	"Let's think step by step..." — ask model to reason	Math, logic, complex reasoning
ReAct	Reason → Act → Observe loop	Agents that use tools
System Prompts	Pre-conversation persona, rules, and constraints	Enterprise apps needing consistent behavior
Self-Consistency	Sample multiple responses, take majority vote	High-stakes reasoning tasks
Tree of Thoughts	Explore multiple reasoning paths simultaneously	Complex problem solving

Fine-Tuning Approaches

Approach	Description	When to Use
Full Fine-	Update all model weights. Expensive, needs many GPUs and large	Large budget, very large

Approach	Description	When to Use
Tuning	dataset	labeled dataset
LoRA	Low-Rank Adaptation — train tiny adapter matrices, freeze base model. 100x fewer params	Custom behavior with limited compute
QLoRA	LoRA + 4-bit quantization. Fits large models on single GPU	Fine-tuning on single A10G/A100 GPU
PEFT	Umbrella term for parameter-efficient methods (LoRA, Adapters, Prefix Tuning)	Efficient customization
RLHF	Reinforcement Learning from Human Feedback — human ratings → reward model → RL fine-tune	Alignment (how ChatGPT was built)

RAG vs Fine-Tuning decision guide:

Need	Choose
Knowledge is frequently updated	RAG
Need citations / traceability	RAG
Large knowledge base (>1M docs)	RAG
Consistent output format / style	Fine-tuning
Domain-specific jargon baked in	Fine-tuning
Fast first iteration	RAG (cheaper to start)
Best practice	Start with RAG. Add fine-tuning if quality bar isn't met.

LLM Evaluation Metrics

Metric	What It Measures	Used For
ROUGE	N-gram overlap between generated and reference text	Summarization
BLEU	Precision of n-gram matches	Machine translation
BERTScore	Semantic similarity using BERT embeddings	General text quality
Perplexity	How well model predicts a test set	Language model quality
Faithfulness	Is answer grounded in context? (no hallucination)	RAG evaluation (RAGAS)
Answer Relevancy	Is answer relevant to the question?	RAG evaluation (RAGAS)

Metric	What It Measures	Used For
Context Precision	What fraction of retrieved chunks were useful?	RAG retrieval quality
Context Recall	Were all relevant chunks retrieved?	RAG retrieval coverage
Human Eval (Likert)	Human raters 1–5 for quality, safety, helpfulness	Production quality gates

Responsible AI & Governance

Topic	Key Points
Hallucination	Model generates confident but wrong facts. Mitigate with RAG grounding, Bedrock Guardrails grounding check, lower temperature
Prompt Injection	Malicious input tries to override system instructions. Mitigate with input validation, Guardrails, sandboxed tool execution
Data Privacy	Bedrock does NOT train on your data. Use VPC endpoints, KMS encryption, PII redaction via Guardrails
Model Bias	Training data biases reflected in outputs. Detect with SageMaker Clarify. Mitigate with diverse data, RLHF
Cost Optimization	Right-size model, use batch inference for offline, provisioned throughput for steady loads, semantic caching
Compliance	CloudTrail for audit logs, GuardDuty for threat detection, AWS Config for compliance rules

6. Common Interview Q&A

Q: When would you use Amazon Bedrock vs SageMaker?

Bedrock when you want to use existing foundation models (Claude, Llama, etc.) without managing infrastructure. Great for RAG apps, chatbots, and enterprise Gen AI that doesn't need custom training. Pay per token, built-in Knowledge Bases, Agents, and Guardrails.

SageMaker when you need to train or fine-tune your own model, require more control over serving infrastructure, use custom PyTorch/TF code, or host open-source models with specific configurations.

Both together: Teams often fine-tune on SageMaker, then use Bedrock's custom model import to bring it into the managed ecosystem.

Q: How would you design a RAG pipeline on AWS for a customer support bot?

Architecture:

Customer Docs/PDFs → S3

→ Bedrock Knowledge Base (auto-chunks, embeds with Titan, stores in OpenSearch Serverless)

User query → API Gateway → Lambda

→ Bedrock Retrieve and Generate API (Claude)

→ Grounded answer with citations → User

Guardrails layer: Bedrock Guardrails for content filtering and PII redaction.

Monitoring: CloudWatch metrics for token usage and latency.

Key design decisions: chunk size (512–1024 tokens), embedding model (Titan vs Cohere), K retrieved chunks (3–5), hybrid search vs pure vector, guardrails configuration.

Q: How do you prevent hallucinations in a production Gen AI app?

Multi-layered approach:

1. **RAG grounding** — retrieve factual context, instruct model to only answer from it
 2. **Bedrock Guardrails grounding check** — auto-detect responses not supported by source docs
 3. **Structured prompts** — "If you don't know, say 'I don't have this information'"
 4. **Temperature control** — lower temperature (0.1–0.3) for factual tasks
 5. **Citation requirements** — require model to cite source passages
 6. **Human-in-the-loop** — for high-stakes decisions
-

Q: How would you optimize cost for a high-traffic Gen AI app on Bedrock?

1. **Right-size the model** — Claude Haiku for simple tasks, Sonnet for complex ones (10x cost difference)
 2. **Provisioned Throughput** — reserve capacity for predictable loads (significant per-token discount)
 3. **Batch Inference** — for offline/async jobs (lowest cost tier)
 4. **Semantic caching** — cache responses for similar queries via Redis, check cosine similarity before calling API
 5. **Prompt caching** — cache repeated system prompts (reduces tokens billed)
 6. **Shorter prompts** — compress system prompts, retrieve only K most relevant chunks
 7. **Cross-Region Inference** — access spare capacity in other regions during peak hours
-

Q: How do you secure a Gen AI application on AWS?

1. **IAM least privilege** — roles for Bedrock with only necessary model access
 2. **VPC private endpoints** — Bedrock VPC endpoints so traffic never leaves the AWS network
 3. **Bedrock Guardrails** — content filtering, PII redaction, prompt injection resistance
 4. **KMS encryption** — encrypt fine-tuned models and Knowledge Base vector stores at rest
 5. **Secrets Manager** — never hardcode API keys, rotate automatically
 6. **CloudTrail** — audit all Bedrock/SageMaker API calls for compliance
 7. **Input validation** — sanitize user input before sending to model
 8. **Data commitment** — Bedrock does not use your prompts for model training (confirm with AWS DPA for regulated industries)
-

Q: What is a Bedrock Agent and how does it work?

A Bedrock Agent is an autonomous AI system that breaks down complex tasks, uses tools, and takes multi-step actions.

Setup: Define action groups (Lambda functions with OpenAPI schemas), optionally attach a Knowledge Base.

ReAct loop at runtime:

1. Receives user task
2. Reasons about what to do (Thought)
3. Decides which action to call (Act)
4. Executes Lambda function
5. Observes result
6. Repeats until task is complete
7. Returns final answer

Example: A customer service agent that checks order status (Lambda → DynamoDB), initiates returns (Lambda → API), and looks up policy (Knowledge Base) — all in one conversation turn.

Q: Explain the difference between fine-tuning and RAG. When to use which?

RAG = retrieval at inference time. No model weight changes. External knowledge injected into prompt. Best for: frequently updated knowledge, large knowledge bases, traceability via citations. Limitation: latency overhead, retrieval quality dependency.

Fine-tuning = modify model weights on your data. Best for: consistent output format/style, domain-specific jargon, classification or extraction tasks. Limitation: expensive, knowledge is static (requires re-training for updates).

Best practice: Start with RAG (cheaper, faster to iterate). If RAG doesn't meet quality bar, fine-tune for style/format, then add RAG on top for up-to-date knowledge. Never fine-tune as a substitute for RAG when knowledge is dynamic.

Q: How would you evaluate the quality of a RAG system?

Use the **RAGAS** framework:

Metric	What It Checks
Faithfulness	Are claims in the answer supported by retrieved context? (hallucination detection)
Answer Relevancy	Is the generated answer actually addressing the question?
Context Precision	What fraction of retrieved chunks were actually useful?
Context Recall	Were all relevant chunks retrieved?
Answer Correctness	End-to-end correctness vs ground truth

On AWS: use **Bedrock Model Evaluation** for systematic comparison, **CloudWatch** dashboards for production metrics, and human evaluation for a random sample (5–10%) of production traffic.

Good luck with your interview tomorrow! 🚀